

דו"ח - תרגיל 1 בינה מלאכותית

חלק 1:

שאלה 1:

מבנה הקובץ `db/israel.csv` הוא של קובץ `csv`, כלומר טבלה כאשר שורות מופרדות על ידי ירידת שורה ועמודות מופרדות על ידי פסיק. מסורתית, וכן בקובץ איתו אנו עובדים, כל שורה מציגת רשומה. ניתן גם לראות זאת בקוד הפונקציה שטוענת את המפה `load_map_from_csv`, כאשר אנו מתייחסים לכל שורה בנפרד ומייצאים אובייקטים שנבנים על פי נתוני שורה אחת בלבד.

כעת אעבור לתיאור המשמעות של העמודות המרכיבות את הרשומות שלנו. הסקה זו קלה לביצוע כאשר רואים שמעבירים לפונקציה `_make_junction` את רכיבי הרשומה השונים. מכאן על פי אינדקסים הרשומות:

0. אינדקס הצומת - מספר ייחודי המשמש כמפתח לצומת.
1. lat (קו רוחב) - קו רוחב גיאוגרפי, מציין את המיקום הפיזי של הצומת בציר האנכי של כדור הארץ.
2. lon (קו אורך) - קו אורך גיאוגרפי, מציין את המיקום הפיזי של הצומת בציר האופקי של כדור הארץ.
3. רשימת הלינקים המחוברים לצומת - כל לינק מוגדר על ידי 3 מספרים מופרדים על ידי סימן ה- '@'. כל אחד מהמספרים מציין את אינדקס צומת היעד, אורך הלינק (במטרים), סוג הלינק בהתאמה. שימו לב שבמבנה הנתונים `Link` אנו מוסיפים מידע מרונדם עבור התנועה בלינק.

חלק 3:

שאלה 8:

בחלק זה עסקנו באלגוריתם A^* . באלגוריתם זה ביצענו חיפוש בגרף תוך כדי שימוש בשתי פונקציות על מנת להתקדם לבחור את הצומת הבאה אליה נתקדם בגרף בעת החיפוש. שתי פונקציות אלה הינן g ו- h (על פי סימוני התרגול). אסביר על שתיהן כאשר אתמקד בפונקציה ההיוריסטית (h), כפי שהתבקשנו בשאלה:

g - פונקציית עלות הדרך שעברתי עד כה. מחושבת במונחי זמן כאשר לצורך חישובה נעשה שימוש

$$\text{time} = \frac{\text{distance}}{\text{velocity}}, \text{ כלומר } STV, \text{ בנוסחת}$$

חשוב לזכור שאנו מפעילים את שתי הפונקציות על הצומת הנוכחית במטרה לשערך את עלות המסלול הכולל שאנחנו נקבל (כאשר משתמשים בצומת הנוכחית כחלק מהמסלול). כלומר, בעת פיתרון בעיית החיפוש, בצומת כלשהי, יש לנו "ביד" מסלול כלשהו מצומת ההתחלה לצומת הנוכחית ולכן אין אנו צריכים לשערך כלום, אלא לחשב בדיוק את העלות של התת-מסלול הזה. ובדיוק כך אנו עושים. אנו לוקחים את המרחקים של כל הקשתות ומחלקים במהירות המקסימלית המותרת לנסיעה בה (כפול 1000 לצורך המרה מק"מ למטר).

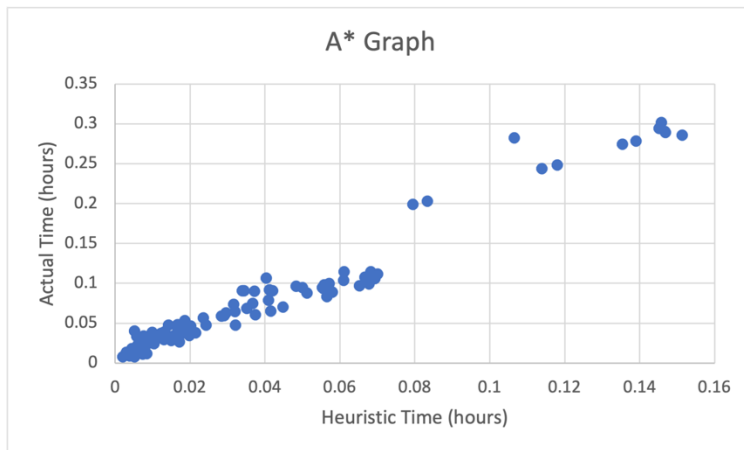
h - פונקציה היוריסטית עבור שארית הדרך. אין אנו יודעים את שאר הצמתים בתת-מסלול מהצומת הנוכחית לצומת המטרה. לכן, אנו נותנים שערוך לעלות מסלול זה על ידי חישוב הזמן לנסיעה כאשר המרחק הינו המרחק האוקלידי בין הצומת הנוכחית למטרה ונחלק במהירות המקסימלית לנסיעה מבין כל הכבישים (110) - עם התאמות יחידות מידה. כך נקבל תמיד את זמן הנסיעה המינימלי האפשרי בעבור תת מסלול זה וכפי שראינו בתרגול בהכרח מדובר בפונקציה היוריסטית שמבטיחה אופטימליות של האלגוריתם. מכאן קל לראות הן שהפונקציה היוריסטית קבילה (תמיד גדולה מאפס וכן קטנה מהעלות האמיתית - מרחק אווירי הכי קצר + מהירות מקסימלית) וכן מונוטונית (שוב ישיר מהגדרת הפונקציה והשימוש במרחק אווירי).

שאלה 9:

אתחיל בלהציג את תהליך יצירת הבעיות (למעשה - חלק 1, שאלה 3): על מנת שלא אגביל את עצמי כלל בבעיות החיפוש אותן אני בוחר להרצות השאלה וככה שאנסה את אלגוריתמי החיפוש שנלמדו על בעיות כמה שיותר כלליות התחלתי בהגרלת מספרי צמתים רנדומליים בטווח האינדקסים של הצמתים מכל צמתי המפה. לאחר מכן, הרצתי את אחד מהאלגוריתמים שנלמדו (ספציפית בפעם האחרונה שג'ינרטי בעיות חיפוש - AStar) על מנת לוודא שהצמתים קשירים וכן שמספר הצמתים בדרך ביניהם גדולה מ-5 (דרישה שהופיעה בפורום שבאתר התרגיל). כמו כן, שמתי timeout על ריצת האלגוריתם עבור כל בעיית חיפוש של 15 שניות (בפועל נדיר שנתקלתי בחסם העליון הזה), על מנת שכאשר הקוד ירוץ במחשבי הבודקים, זמן ההרצה לא יהיה מוגזם.

כעת נעבור להסתכל על הגרף וניתוחו:

כל נקודה מציינת בעיית חיפוש שונה, כאשר ערכה על ציר x הוא הזמן ההיוריסטי (בשעות) של נסיעה מצומת ההתחלה לצומת היעד. הערך על ציר ה-y הוא הזמן האמיתי (בשעות) של אותה נסיעה, כלומר במסלול האופטימלי שהאלגוריתם הוציא כפלט. אנו רואים בגרף מגמה ליניארית מובהקת.



כמו כן, ישנם מספר סימנים המעידים כי הפונקציה ההיוריסטית שלנו הינה טובה. ראשית, ניתן לראות מאוד בבירור כי העלות ההיוריסטית חסומה על ידי העלות האמיתית. יתרה מכך, הגרף מציג מגמה ליניארית שמראה יחס טוב בין העלות המשוערכת לאמיתית ושככל שהעלות האמיתית גדלה כך גם העלות ההיוריסטית של המסלול. זאת ועוד, כפי שנלמד בתרגול, פונקציה היוריסטית טובה נעה בין חצי מהעלות האמיתית לעלות האמיתית (כשהמטרה כמובן להיות כמה שיותר קרובים למטרה). אכן הפונקציה ההיוריסטית שלנו מקיימת אי שוויון זה, אם כי היא קרובה יחסית לצד של החצי באי שוויון המדובר.

שאלה 10:

במונחי ביצועים של העולם האמיתי (כפי שרומזת השאלה), במידה והרצנו את האלגוריתם לפני תחילת שעת העומס ונבצע את הנסיעה שלנו באותו מסלול בשעת העומס. ככל הנראה המסלול לא יהיה אופטימלי.

הסיבה לכך הינה שהפונקציה ההיוריסטית בה השתמשנו לא מתחשבת כלל בתנאי הדרך. מכאן שכאשר תתחיל שעת העומס, יכול להיות שמהירות הנסיעה בכביש שמהווה חלק מהמסלול תרד למהירות נמוכה מספיק כך שהמסלול האופטימלי כעת הוא לקחת מסלול שעוקף את אותו הכביש, למרות שאותו כביש מהווה חלק מהמסלול הקצר ביותר.

הדבר מוקצן הרבה פעמים בשעות העומס כאשר ההערכה של אפליקציית ניווט בכבישים לוקה בחסר בעבור זמן מסוים ולכן היא מנחה לקחת מסלול מסוימת בשביל להגיע ליעד, אך משום שהיא לא התחשבה כראוי בעומס באותה שעה, בהנחה ורבים משתמשים באפליקציה, נחוזה "תופעת עדר" והם יסעו באותה הדרך ורק יגבירו את העומס באותו הכביש. דרכים אפשריות למיטיגציה של בעיה זו הינה עדכון המסלול תוך כדי ההילוך בו (תוך כדי הנסיעה), ביזור של הרכבים השונים בדרכים שונות בעלות זמן נסיעה כולל דומה וכן שימוש בפונקציה ההיוריסטית בכמה שיותר מידע לגבי העומס בכבישים במהלך שעות היום.

חלק 5:

שאלה 12:

יש לשים לב בעת הרצה כי שמות הקבצים מציניים את מספר השורה בקובץ problems.csv עבורו ייצאנו את מפת המסלול (וכן שאנו מתחילים לספור את השורות מ-0).

שאלה 13:

רצוי לחזור לשאלה 9, שכן שם תיארתי את האופן בו יצרתי את בעיות החיפוש. אציין בקצרה בשנית כי הקו המנחה היה שבעיית החיפוש תהיה כמה שיותר כללית. את הממוצע חישבתי על כל 100 בעיות החיפוש שבקובץ המצורף, דבר שמרמז את התוצאות אותן אסביר בהמשך.

ממוצעי האלגוריתמים הינם:

- אלגוריתם UCS - 1.939055 שניות.
- אלגוריתם AStar - 1.605046 שניות.
- אלגוריתם IDAStar - מעל ל-12 שעות.

יתרה מכך, מצאתי את זה מעניין שעבור UCS שונות התוצאות הייתה 0.405 ואילו עבור AStar היא הייתה 0.34855.

ניתן לראות שמבין האלגוריתמים שנבדקו, AStar נתן לנו את התוצאה הטובה ביותר. התוצאה די צפויה משום שאנו משתמשים במידע נוסף על המידע בה משתמש אלגוריתם UCS על מנת לחזות באופן יותר מדויק את הקשת הבאה שעליה עלינו להלך על מנת להגיע ליעד בעלות המינימלית.

כפי שהצגתי בשאלות קודמות, הפונקציה ההיוריסטית (אותה פונקציה שעוזרת לנו עם התחזית), היא פונקציה היוריסטית טובה (למרות שבשאלה 10 ציינתי לא מעט דרכים לשפר אותה) והדבר ניכר בתוצאות.

תוצאת אלגוריתם IDAStar אינה טובה והוא בעל הזמן הארוך ביותר משתי סיבות. הראשונה - התבקשנו להשתמש באלגוריתם כאילו שהגרף עליו אנו עובדים הוא עץ. הדבר הוביל לכך שלא בדקנו אם ביקרנו כבר בעבר בצומת מסוימת ושנכנסנו ללא מעט לולאות. כמובן שכפי שפועל שנלמד בשיעור, בסופו של דבר נצא מהלולאה משום שהמסלול האופטימלי באופן טריוויאלי אינו עובר שם וכן כפי שפועל האלגוריתם, העלות הנוספת של מעבר בלולאה במעבר מסוים תעבור את העלות של לעבור בקשת אחרת (לפי חישובי ה- f_limit).

השניה - האלגוריתם במעבר לצומת כלשהו, יהלך בשנית על כל המסלול עד לאותה הצומת החדשה. שתי עובדות אלה יוצרות תקורת זמן ריצה משמעותית מאוד.